

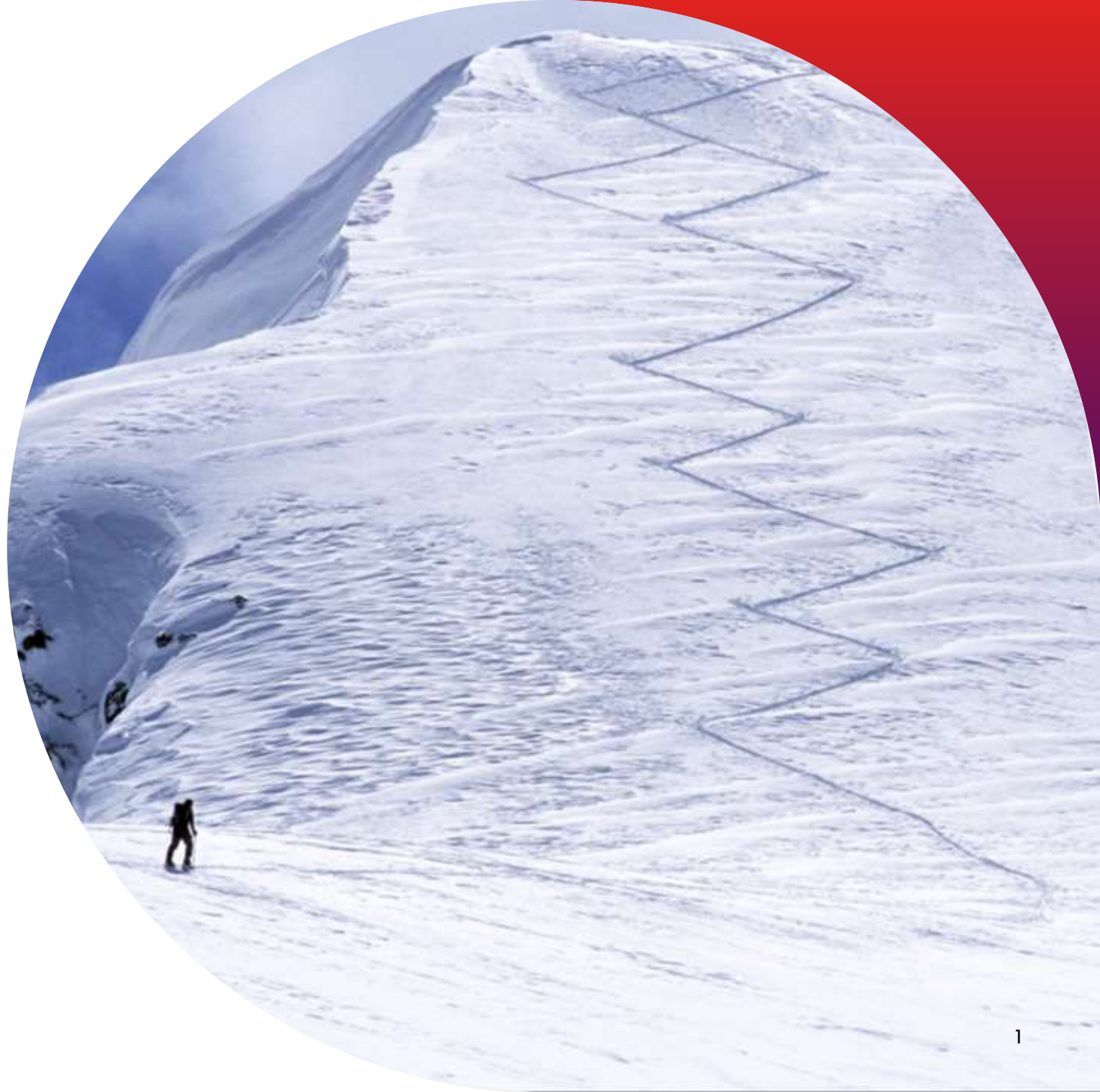
PARIS - CACHAN . TROYES . MONTPELLIER . SAINT-NAZAIRE . DAKAR

Numerical optimization for machine learning

T. Drumond



ÉCOLE D'INGÉNIEUR·E·S
Creating the future together



Exercise 1

Machine learning case



Exercise 1.1

You want to learn a linear regression predictor of the form

$$f_{\boldsymbol{\theta}}(\mathbf{x}_i) = \boldsymbol{\theta}^T \mathbf{x}_i$$

where $\boldsymbol{\theta} \in \mathbb{R}^n$, $\mathbf{x}_i \in \mathbb{R}^n$, $f_{\boldsymbol{\theta}}(\mathbf{x}_i) \in \mathbb{R}$.

You have a training set with M samples:

$$\mathcal{D}_{train} = \{(\mathbf{x}_i, y_i)\}_{i=1}^M, \text{ where } y_i \in \mathbb{R}$$

You will use the least squares loss function. For each data sample the loss is:

$$\ell(y_i; f_{\boldsymbol{\theta}}(\mathbf{x}_i)) = (y_i - f_{\boldsymbol{\theta}}(\mathbf{x}_i))^2$$

1. Write the empirical loss for this problem:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{M} \sum_{i=1}^M \ell(y_i, f_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

Is this function convex over $\boldsymbol{\theta}$? Tip: it is

possible to know it without calculating the gradient or the hessian.

2. Write the gradient of this cost function w.r.t. the optimization parameters
3. Let $\mathbf{y} = [y_1 \ y_2 \ \cdots \ y_M]^T$ be a vector collecting all targets y_n , and $\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_M]^T$ a matrix collecting all input data vectors. Rewrite the cost function and its gradient in terms of $\mathbf{X}$ and $\mathbf{y}$
4. Starting from the vectorized form of the gradient, find the minimizer of this cost function analytically

Derivatives in higher dimensions

Vector-valued functions and Jacobian

n-dim vector $\mathbb{R}^n$	Vector in $\mathbb{R}^n$ $\mathbf{x} = [x_1 \quad \cdots \quad x_n]^T$	Gradient of a real-valued function $\nabla f_1(\mathbf{x}) = \frac{\partial f_1(\mathbf{x})}{\partial \mathbf{x}} = \left(\frac{\partial f_1}{\partial x_1} \quad \cdots \quad \frac{\partial f_1}{\partial x_n} \right)^T$
m-dim vector $\mathbb{R}^m$	Vector-valued function $\mathbf{f}(\mathbf{x}) = [f_1(\mathbf{x}) \quad \cdots \quad f_m(\mathbf{x})]^T$	Partial derivative of a vector-valued function $\frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_j} = \left(\frac{\partial f_1}{\partial x_j} \quad \cdots \quad \frac{\partial f_m}{\partial x_j} \right)^T$

Jacobian matrix

“Gradient” of a vector-valued function

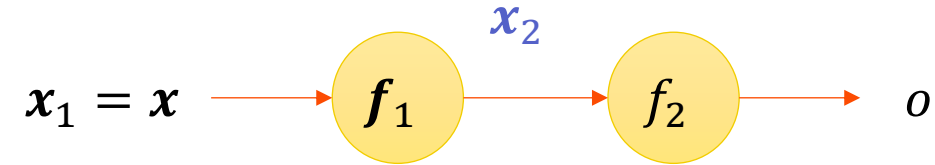
$$J_f(\mathbf{x}) = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}^T} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\mathbf{x})^T \\ \vdots \\ \nabla f_m(\mathbf{x})^T \end{pmatrix} = \left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_1} \quad \cdots \quad \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_n} \right) \in \mathbb{R}^{m \times n}$$

Derivatives in higher dimensions

Gradient for a function composition

$$o = f(\mathbf{x}) \quad \left\{ \begin{array}{l} \mathbf{x} \in \mathbb{R}^n \\ o \in \mathbb{R} \end{array} \right.$$

$$f = f_2 \circ f_1$$



$$\begin{aligned} \mathbf{x}_2 &= f_1(\mathbf{x}_1) \in \mathbb{R}^m \\ o &= f_2(\mathbf{x}_2) \in \mathbb{R} \end{aligned}$$

$$\frac{\partial o}{\partial \mathbf{x}} = \frac{\partial o}{\partial \mathbf{x}_2^T} \frac{\partial \mathbf{x}_2}{\partial \mathbf{x}_1^T} = \frac{\partial f_2(\mathbf{x}_2)}{\partial \mathbf{x}_2^T} \frac{\partial f_1(\mathbf{x})}{\partial \mathbf{x}^T} = \nabla f_2(\mathbf{x}_2)^T \mathbf{J}_{f_1}(\mathbf{x})$$

Exercise 1.2

You want to build a binary classifier using a logistic regression model of the form

$$f_{\boldsymbol{\theta}}(\mathbf{x}_i) = \frac{1}{1+e^{-\boldsymbol{\theta}^T \mathbf{x}_i}}$$

where $\boldsymbol{\theta} \in \mathbb{R}^n$, $\mathbf{x}_i \in \mathbb{R}^n$, $f_{\boldsymbol{\theta}}(\mathbf{x}_i) \in [0,1]$.

You have a training set with M samples:

$$\mathcal{D}_{train} = \{(\mathbf{x}_i, y_i)\}_{i=1}^M, \text{ where } y_i \in \{0,1\}$$

You will use the cross-entropy loss function. For each data sample the loss is:

$$\begin{aligned} \ell(y_i; f_{\boldsymbol{\theta}}(\mathbf{x}_i)) = \\ -y_i \log f_{\boldsymbol{\theta}}(\mathbf{x}_i) + (1 - y_i) \log(1 - f_{\boldsymbol{\theta}}(\mathbf{x}_i)) \end{aligned}$$

1. Write the empirical loss for this problem:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{M} \sum_{i=1}^M \ell(y_i, f_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

Is this function convex over $\boldsymbol{\theta}$?

2. Write the gradient of this cost function w.r.t. the optimization parameters
3. Let $\mathbf{y} = [y_1 \ y_2 \ \cdots \ y_M]^T$ be a vector collecting all targets y_n , and $\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_M]^T$ a matrix collecting all input data vectors. Rewrite the cost function and its gradient in terms of $\mathbf{X}$ and $\mathbf{y}$
4. Starting from the vectorized form of the gradient, find the minimizer of this cost function analytically

Derivatives in higher dimensions

Gradient for a function composition

$$\mathbf{x}_2 = f_1(\mathbf{x}_1) \in \mathbb{R}^m$$

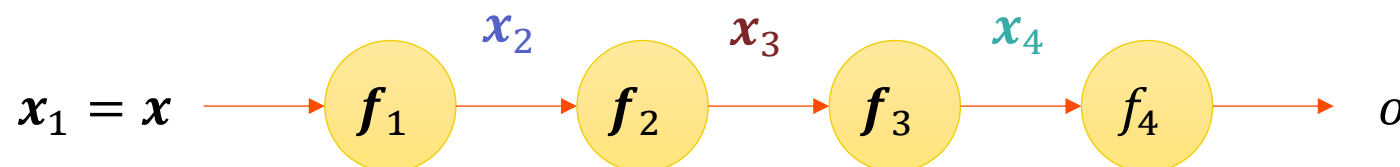
$$\mathbf{x}_3 = f_2(\mathbf{x}_2) \in \mathbb{R}^p$$

$$\mathbf{x}_4 = f_3(\mathbf{x}_3) \in \mathbb{R}^q$$

$$o = f_4(\mathbf{x}_4) \in \mathbb{R}$$

$$o = f(\mathbf{x}) \begin{cases} \mathbf{x} \in \mathbb{R}^n \\ o \in \mathbb{R} \end{cases}$$

$$f = f_4 \circ f_3 \circ f_2 \circ f_1$$



$$\begin{aligned} \frac{\partial o}{\partial \mathbf{x}} &= \frac{\partial o}{\partial \mathbf{x}_4^T} \frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3^T} \frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2^T} \frac{\partial \mathbf{x}_2}{\partial \mathbf{x}_1^T} = \frac{\partial f_4(\mathbf{x}_4)}{\partial \mathbf{x}_4^T} \frac{\partial f_3(\mathbf{x}_3)}{\partial \mathbf{x}_3^T} \frac{\partial f_2(\mathbf{x}_2)}{\partial \mathbf{x}_2^T} \frac{\partial f_1(\mathbf{x})}{\partial \mathbf{x}^T} \\ &= \nabla f_4(\mathbf{x}_4)^T \mathbf{J}_{f_3}(\mathbf{x}_3) \mathbf{J}_{f_2}(\mathbf{x}_2) \mathbf{J}_{f_1}(\mathbf{x}) \end{aligned}$$

Exercise 1.3

You want to learn a multi-output linear regression of the form $\mathbf{f}_\theta(\mathbf{x}) = \mathbf{W}\mathbf{x}$, where $\mathbf{f}_\theta(\mathbf{x}) \in \mathbb{R}^m$, $\theta = \mathbf{W} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$.

You will use the least squares cost function: $\mathcal{L}(\theta) =$

$$\frac{1}{N} \sum_{i=1}^N \|\mathbf{y}_i - \mathbf{f}_\theta(\mathbf{x}_i)\|^2$$

Write the gradient of this cost function w.r.t. the optimization parameters $\mathbf{W}$

- a) First compute $\frac{\partial \mathcal{L}}{\partial \mathbf{f}}$, it should be an $m \times 1$ vector composed of each $\frac{\partial \mathcal{L}}{\partial f_k}$
- b) Then compute $\frac{\partial \mathbf{f}}{\partial W_{ij}}$, it should be a Jacobian $m \times 1$ vector composed of each $\frac{\partial \mathcal{L}}{\partial f_k}$ (you will need to relate elements f_k of the vector $\mathbf{f}$ and elements W_{ij} of matrix $\mathbf{W}$)

Note: since $\mathbf{W}$ is $m \times n$, and each $\frac{\partial \mathbf{f}}{\partial W_{ij}}$ is $m \times 1$, $\frac{\partial \mathbf{f}}{\partial \mathbf{W}}$ can be represented as an $m \times (m \times n)$ matrix.

- c) Let $\mathbf{u}$ be a $m \times 1$ vector. Show that $\mathbf{u}^T \frac{\partial \mathbf{f}}{\partial W_{ij}} = u_i x^j \in \mathbb{R}$

Note: the product $\mathbf{u}^T \frac{\partial \mathbf{f}}{\partial \mathbf{W}}$ can be represented by a $1 \times (m \times n)$ matrix. To simplify notation, let's represent it by an $m \times n$ matrix.

- d) Show that $\mathbf{u}^T \frac{\partial \mathbf{f}}{\partial \mathbf{W}} = \mathbf{u} \mathbf{x}^T$
- e) Finally, by making $\mathbf{u} = \frac{\partial \mathcal{L}}{\partial \mathbf{f}}$, compute $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{f}} \frac{\partial \mathbf{f}}{\partial \mathbf{W}}$ (it should be a matrix $m \times n$)

Exercise 2

GD with momentum



Gradient descent with momentum

Parameter update

$$\theta_{t+1} = \theta_t - \rho_t \mathbf{m}_{t+1}$$

Where $\mathbf{m}_t$ is the momentum at time t

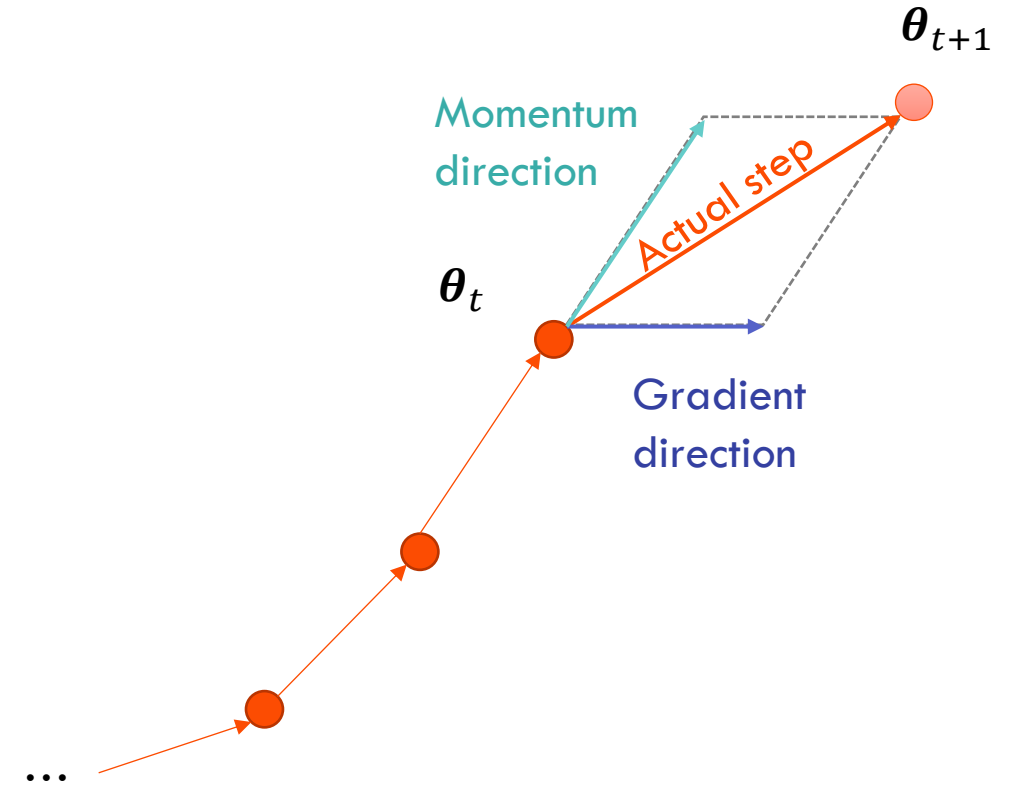
Momentum

Exponentially weighted moving average of the past gradients

$$\mathbf{m}_{t+1} = \beta \mathbf{m}_t + \mathbf{g}_t$$

Where $0 < \beta < 1$ is the **decay rate**

Typical default value: $\beta = 0.9$



Exercise 2.1

1. Develop the momentum recurrence equation

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + \mathbf{g}_{t-1}$$

to show that it represents an exponentially-weighted moving average (EWMA) of the past gradients. That is, show that

$$\mathbf{m}_t = \sum_{\tau=0}^{t-1} \beta^\tau \mathbf{g}_{t-\tau-1}$$

2. Consider the past gradients to be all constant and equal to $\mathbf{g}$. In this case, show that for $t \rightarrow \infty$ momentum accelerates the gradient by a factor $\alpha > 1$. That is, show that

$$\lim_{t \rightarrow \infty} \mathbf{m}_t = \alpha \mathbf{g}$$

with $\alpha > 1$. Express α in terms of β .

Exercise 2.2

Go to **G. Goh, 2017: Why Momentum Really Works (distill.pub)**

You are going to explore the visualization provided at the top of the article.

1. Set step size α to 0.003 (center of the slider). Then set the momentum decay rate β to the following values and observe the path taken during the optimization
 - a) 0.00 (no momentum, pure GD): Lots of oscillations at the beginning, then steady convergence
 - b) 0.50: oscillations are dampened and optimization progresses further within the same number of iterations
 - c) 0.90: You will observe some oscillations (instability) when approaching the minimum
 - d) Find a good value for β under which the minimum is reached without oscillations near it.
2. Now that you found a good decay rate, change the step size. Is this decay rate still appropriate? How far can you move α without needing to change it?

Exercise 3
GD with Nesterov's
momentum



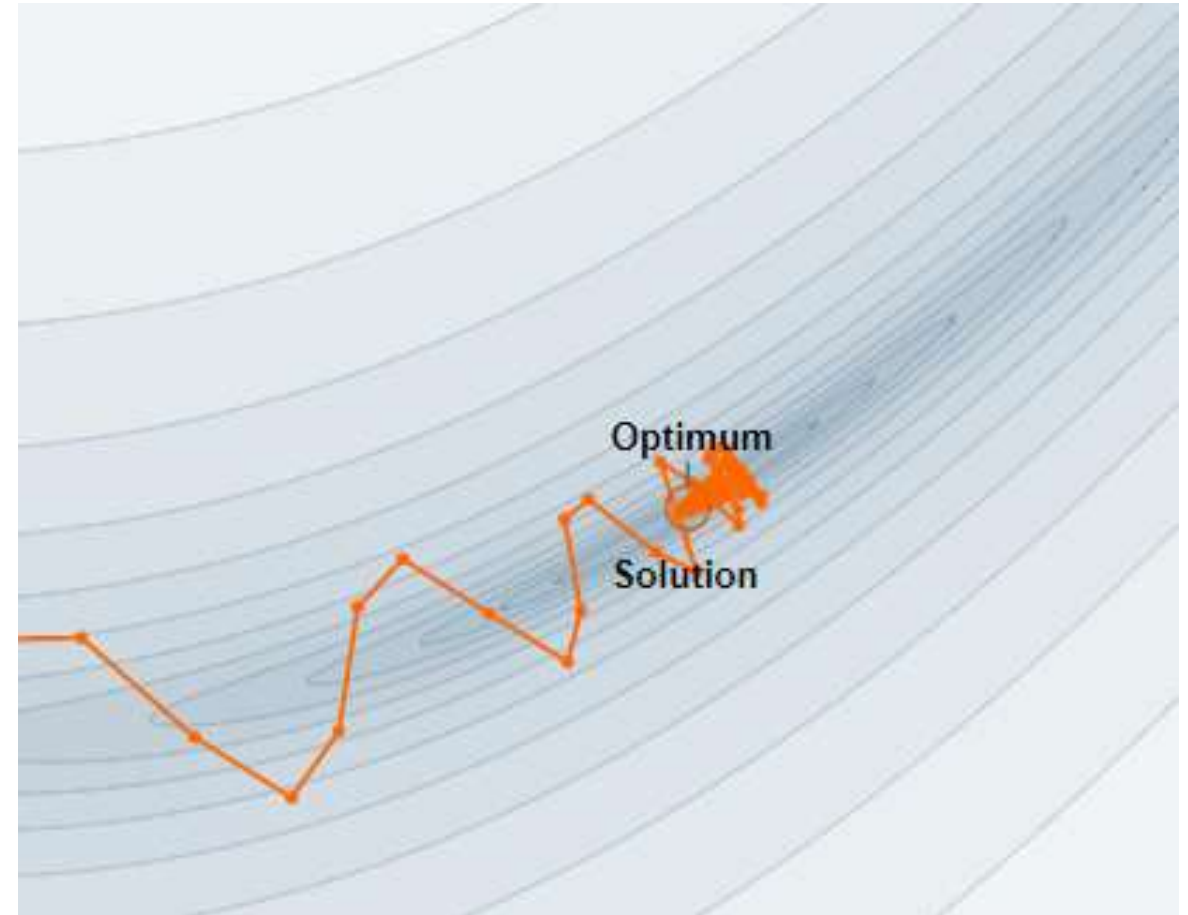
Nesterov accelerated gradient

Problem

Regular momentum may over-accelerate and cause **oscillations** when approaching the minimum

Solution

Take momentum step before the gradient step
→ **Gradient computation is more locally accurate**



Nesterov accelerated gradient

Nesterov acceleration

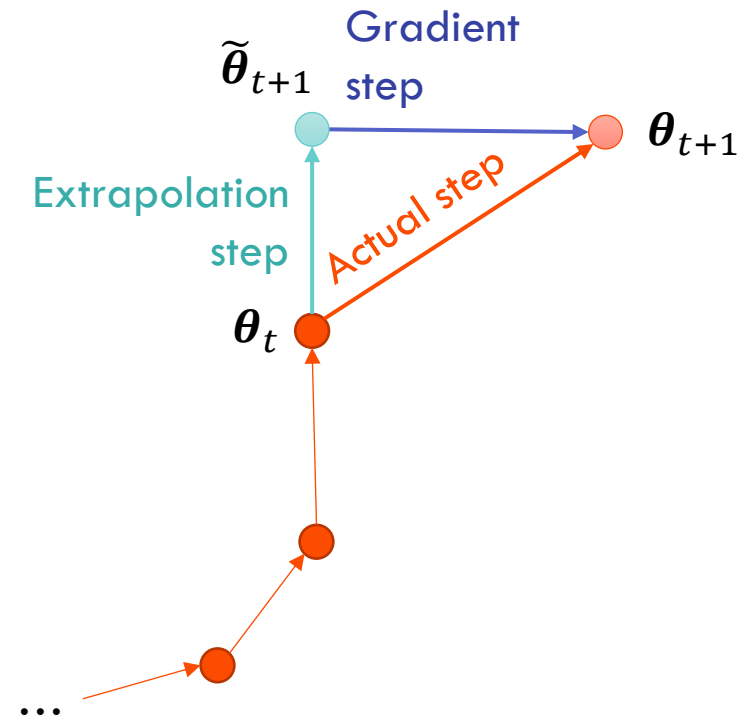
Let $\Delta\theta_t = \theta_t - \theta_{t-1}$ be the last step

1. Extrapolate an intermediate next position $\tilde{\theta}_{t+1}$ from the last step

$$\tilde{\theta}_{t+1} = \theta_t + \beta \Delta\theta_t$$

2. From there, make a gradient update

$$\theta_{t+1} = \tilde{\theta}_{t+1} - \rho_t g(\tilde{\theta}_{t+1})$$



Exercise 3

Note: For this exercise we will consider a constant learning rate $\rho_t = \rho$.

Consider the following reformulation of GD with momentum:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \tilde{m}_{t+1} \\ \tilde{m}_{t+1} &= \underbrace{\beta \tilde{m}_t - \rho g(\theta_t)}_{\text{descent step}}\end{aligned}$$

Rewrite the Nesterov acceleration equations so that they get presented in a similar way. That is, rewrite them so that the **descent step** is composed by a **momentum term** + a **gradient term**.

Tip: for the Nesterov case, define $\tilde{m}_t := \Delta\theta_t = \theta_t - \theta_{t-1}$

What is the main difference you observe between both methods?

Nesterov accelerated gradient

Convergence

In theory

Faster than GD on convex functions
(for appropriate β_t and ρ_t)

Optimal convergence for convex cost with
smooth gradient (within 1st order methods)

In practice

May become **unstable** if β_t and ρ_t are
mispecified

Exercise 4

GD with preconditioning



Preconditioned SGD

Accelerating SGD convergence

Preconditioning matrix

$$\theta_{t+1} = \theta_t - \rho_t \mathbf{M}_t^{-1} \mathbf{g}_t$$

Often a diagonal matrix

$$\mathbf{M}_t = \text{diag}(\boldsymbol{\eta}_t)$$

So that $\mathbf{M}_t^{-1} = \text{diag}\left(\frac{1}{\boldsymbol{\eta}_t}\right)$

Update step for dimension d becomes:

$$\theta_{t+1}^{(d)} = \theta_t^{(d)} - \rho_t \frac{1}{\eta_t^{(d)}} g_t^{(d)}$$

The term $\eta_t^{(d)}$ **approximates local curvature** by keeping a **memory of past gradients** in the direction d

Preconditioned SGD

Accelerating SGD convergence

$$\theta_{t+1}^{(d)} = \theta_t^{(d)} - \rho_t \frac{1}{\eta_t^{(d)}} g_t^{(d)}$$

The term $\eta_t^{(d)}$ keeps a memory of gradients and in the direction d

Effect

Normalize current gradient w.r.t. past gradients

Direction d has been having **large gradients**

→ Cost is very steep in this direction → move **slower** along it

Direction d has been having **small gradients**

→ Cost is flat in this direction → move **faster** along it

Exercise 4.1

The next slides will present the update steps used by different preconditioning methods. From their respective formulations extract the equivalent preconditioning term $\eta_t^{(d)}$ and fill up the following table.

Method	Preconditioning term $\eta_t^{(d)}$	$\eta_t^{(d)}$ depends on mov. avg. of gradients ²	$\eta_t^{(d)}$ depends on mov. avg. of past updates ²	Update includes a momentum term
AdaGrad	Expression?	yes / no	yes / no	yes / no
RMSprop	Expression?	yes / no	yes / no	yes / no
AdaDelta	Expression?	yes / no	yes / no	yes / no
Adam	Expression?	yes / no	yes / no	yes / no

AdaGrad

Idea: normalize gradient with the gradient memory

Keep track of past gradients via a **squared sum** term:

$$s_t^{(d)} = \sum_{\tau=1}^t \left(g_{\tau}^{(d)} \right)^2$$

where $g^{(d)} = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \theta_d}$, $\boldsymbol{\theta} \in \mathbb{R}^D$, i.e., the d -th component of the gradient vector $\mathbf{g} \in \mathbb{R}^D$

Update step becomes:

$$\theta_{t+1}^{(d)} = \theta_t^{(d)} - \rho_t \frac{1}{\sqrt{s_t^{(d)} + \epsilon}} g_t^{(d)}$$

Included for
numerical
stability

Or in vectorized form:

$$\Delta \boldsymbol{\theta}_t = -\rho_t \frac{1}{\sqrt{\mathbf{s}_t + \epsilon}} \mathbf{g}_t$$

(square root taken element-wise)

The **preconditioning matrix** then is:

$$\mathbf{M}_t = \text{diag}(\mathbf{s}_t + \epsilon)^{\frac{1}{2}}$$

RMSprop

Similar to AdaGrad but different $\mathbf{s}_t$

$$s_{t+1}^{(d)} = \beta s_t^{(d)} + (1 - \beta) \left(g_t^{(d)}\right)^2$$

This is an **exponentially-weighted moving average (EWMA)** of the past gradient values

Usually, $\beta = 0.9$. In this case we have:

$$\sqrt{s_t^{(d)}} \approx RMS \left(g_{1:t}^{(d)}\right) = \sqrt{\frac{1}{t} \sum_{\tau=1}^t \left(g_{\tau}^{(d)}\right)^2}$$

Same form for update step and preconditioning matrix as AdaGrad

AdaDelta

Similar to RMSprop but **also keeps track of last updates** with another EWMA term:

$$\delta_t = \beta \delta_{t-1} + (1 - \beta)(\Delta \theta_t)^2$$

where $(\Delta \theta_t)^2 = \Delta \theta_t \circ \Delta \theta_t$ (elementwise product)

Similarly to momentum, we want to move faster along past movement directions

→ Learning rate is **reduced with gradient memory** and **increased with update memory**

Update step becomes

$$\Delta \theta_t = -\rho_t \frac{\sqrt{\delta_{t-1} + \epsilon}}{\sqrt{s_t + \epsilon}} g_t$$

Preconditioning matrix

$$M_t = \text{diag} \left(\frac{s_t + \epsilon}{\delta_{t-1} + \epsilon} \right)^{\frac{1}{2}}$$

Adam

Similar to RMSprop, but includes momentum

Momentum term

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

Gradient memory term

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

where $\mathbf{g}_t^2 = \mathbf{g}_t \circ \mathbf{g}_t$ (elementwise product)

Typical values are $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-6}$

Problem of EWMA estimates

Initial estimates of $\mathbf{m}_t$ and $\mathbf{s}_t$ are **biased towards their initial values** $\mathbf{m}_0$ and $\mathbf{s}_0$

In particular, small values when $\mathbf{m}_0 = \mathbf{s}_0 = 0$

Adam's solution

Bias-correction on EWMA estimates:

$$\tilde{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \text{ and } \tilde{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}$$

Before the update step:

$$\Delta \theta_t = -\rho_t \frac{1}{\sqrt{\tilde{\mathbf{s}}_t + \epsilon}} \tilde{\mathbf{m}}_t$$

Exercise 4.2

1. Show that AdaGrad tends to have very small effective learning rates as t grows.
Tip: take a look at its preconditioning term and show that it never decreases with increasing t

2. Show that $s_{t+1}^{(d)} = \beta s_t^{(d)} + (1 - \beta) \left(g_t^{(d)}\right)^2$ defines an EWMA over g_t , that is,

$$s_t = \sum_{\tau=0}^t \beta^\tau g_{t-\tau}$$

3. Imagine along the d -th component, the past gradient values are all equal to g . In this case,
 - a) Show that AdaGrad preconditioner $s_t^{1/2}$ scales down the gradient term $g_t^{(d)}$ by a factor $\frac{1}{g\sqrt{t}}$
 - b) Show that RMSprop preconditioner $s_t^{1/2}$ scales down the gradient term $g_t^{(d)}$ by a factor $\frac{1}{(1-\beta^{t+1})g}$
 - c) What happens to these factors as $t \rightarrow \infty$?